

Graphs and Search in Artificial Intelligence

Dr. Huseyin Kusetogullari

Outline

- Solving problems by searching
- Types of Search Algorithms
- Depth first search algorithm
- Breadth search algorithm
- A* algorithm

Search problems

- Problem-Solving agents
- Problem types
- Problem formulation
- Example problems
- Search algortihms

Important questions

- How much time?
  - Time complexity
- How much space?
  - Space complexity
- Will it always find a solution?
  - Completeness
- Will it always find the best solution?
  - Optimality

Graph Search

- Graph search in AI refers to a set of techniques used to traverse or explore graphs in order to find specific nodes, paths, or structures that meet certain criteria.

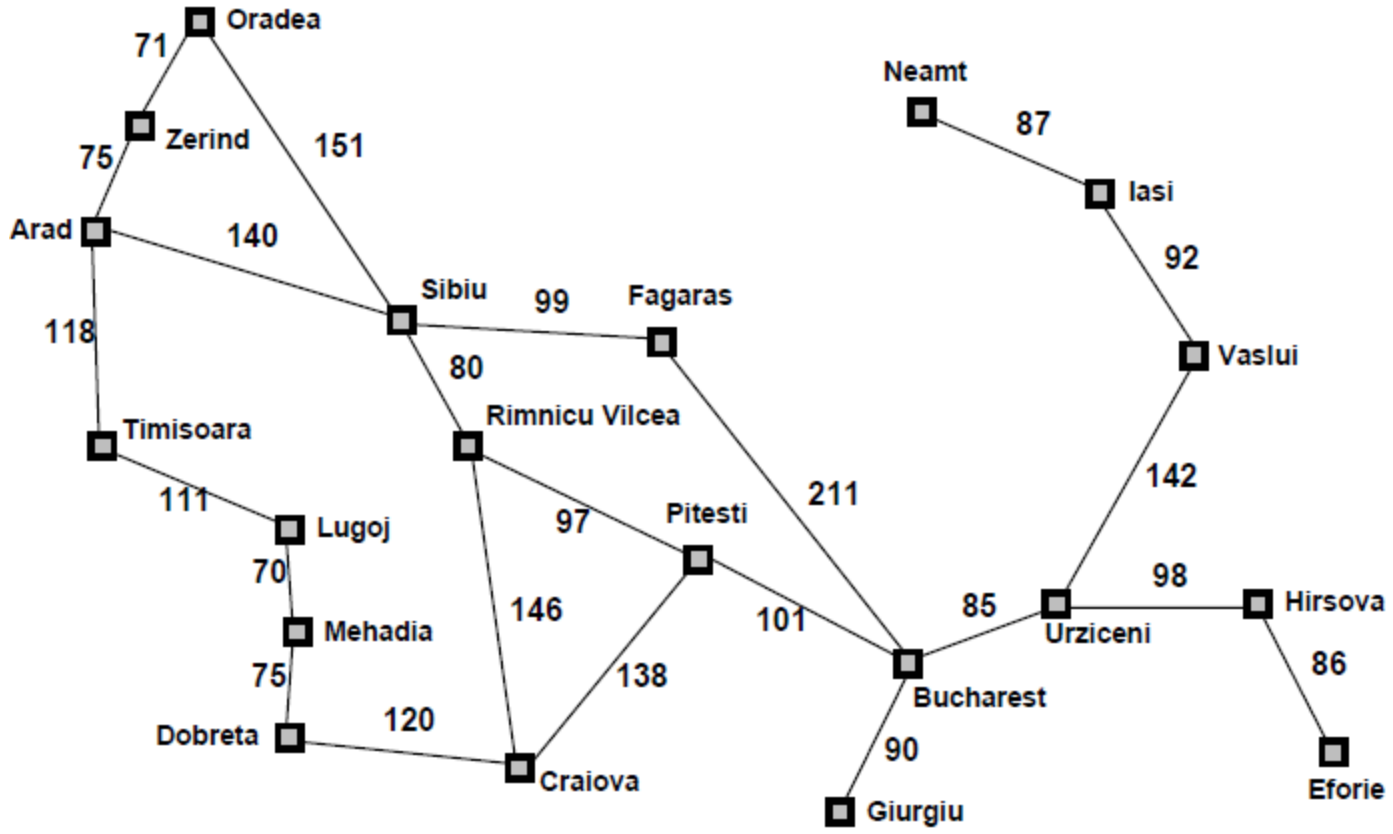
- Graphs are data structures consisting of nodes (also called vertices) and edges that connect pairs of nodes.

- In the context of AI, graph search algorithms are fundamental in solving a variety of problems, such as pathfinding, network analysis, game playing, and knowledge representation.

Example Problems

- Bucharest
- Vacuum agent
- Vacuum world state space graph
- 8-puzzle
- 8-queens
- Route finding
- Traveling salesman
- Robot navigation
- Web search

Example: Bucharest



Initial state: Arad
Goal state: Bucharest

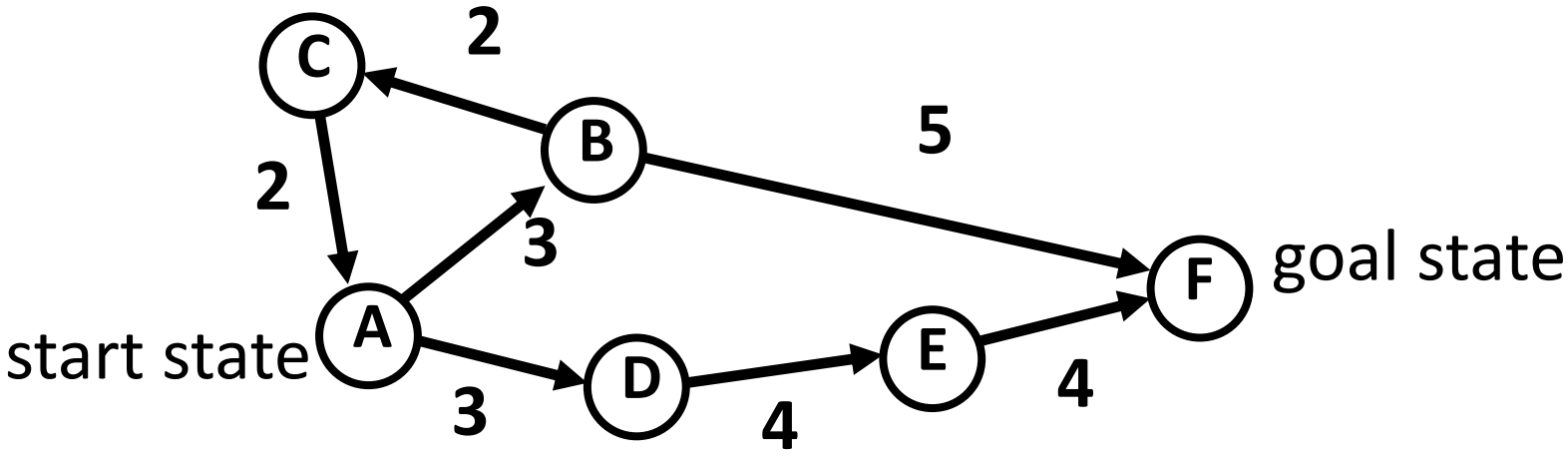
Cont.

- On holiday in Romania; currently in Arad.
- Car is going by tomorrow

- Formulate goal:
  - Be in Bucharest
- Formulate problem:
  - States: various cities
  - Actions: drive between cities

- Find solution: sequence of cities, e.g., Arad, Sibiu, Fagaras, Bucharest

A simple example: traveling on a graph

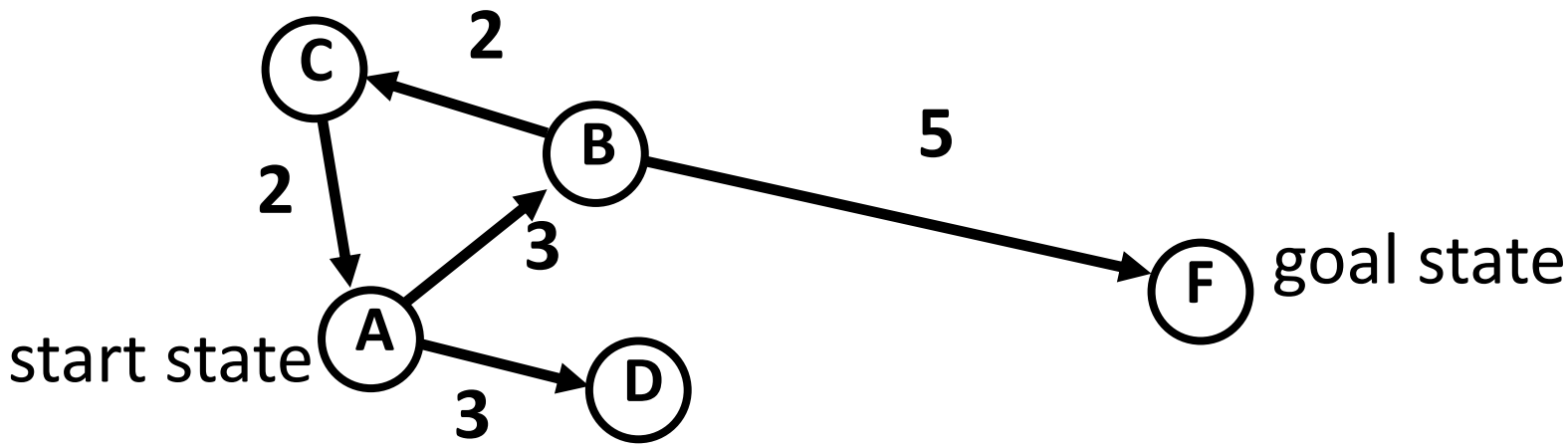


State:

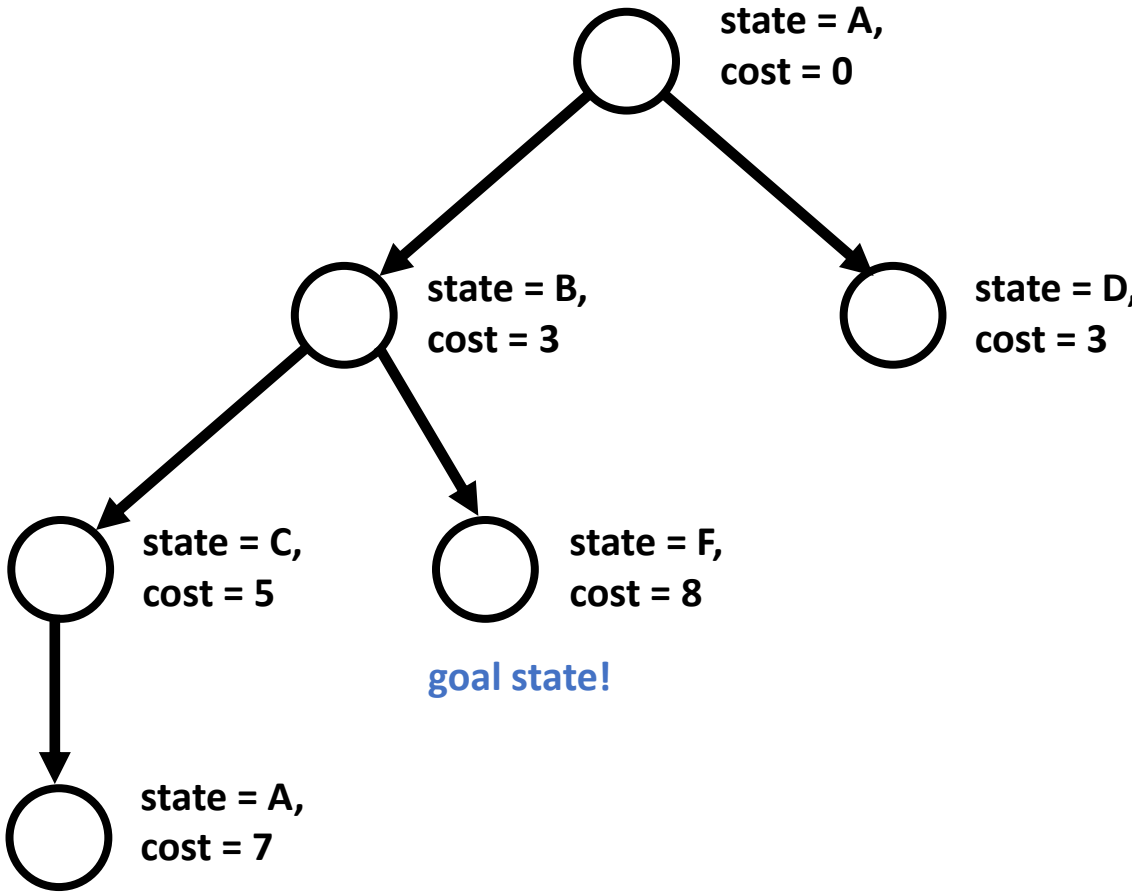
•**Definition:** A state represents a specific configuration or condition of the problem at a given point in time. It is an abstract representation of a possible situation that the system can be in.

•**Example:** In a pathfinding problem, a state could represent a particular position of an agent on a grid.

Searching for a solution

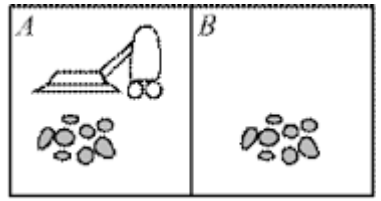


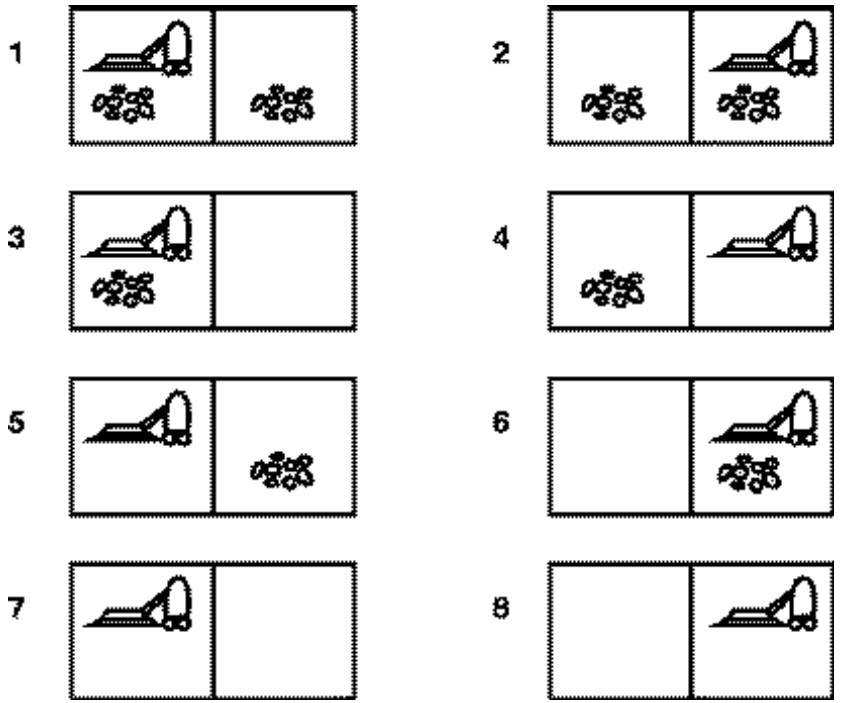
Search tree


state = A,
cost = 0
state = B,
cost = 3
state = D,
cost = 3
state = C,
cost = 5
state = F,
cost = 8
goal state!
state = A,
cost = 7

Example: Vacuum world

- Imagine that our intelligent agent is a robot vacuum cleaner. Let's suppose that the world has just two rooms. The robot can be in either room and there can be dirt in zero, one, or two rooms.
- **Goal formulation:** intuitively, we want all the dirt cleaned up. Formally, the goal is **{ state 7, state 8 }.**





There are 8 states

Problem-Solving Agents

- Agents whose task it is to solve a particular problem
  - Goal formulation
    - What is the goal state
    - What are the important characteristics of the goal state
    - How does the agent know that it has reached the goal
    - Are there several possible goal states
- Problem formulation
  - What are the possible states of the world relevant for solving the problem
  - What information is accessible to the agent
  - How can the agent progress from state to state

Well-defined problems

- Problems with a readily available formal specification
  - Initial state
    - Starting point from the which the agent sets out
  - Actions (operators, successor functions)
    - Describe the set of possible actions
  - State space
    - Set of all states reachable from the initial state by any sequence of actions
  - Path
    - Sequence of actions leading from one state in the state space to another
  - Goal test
    - Determines if a given state is the goal state

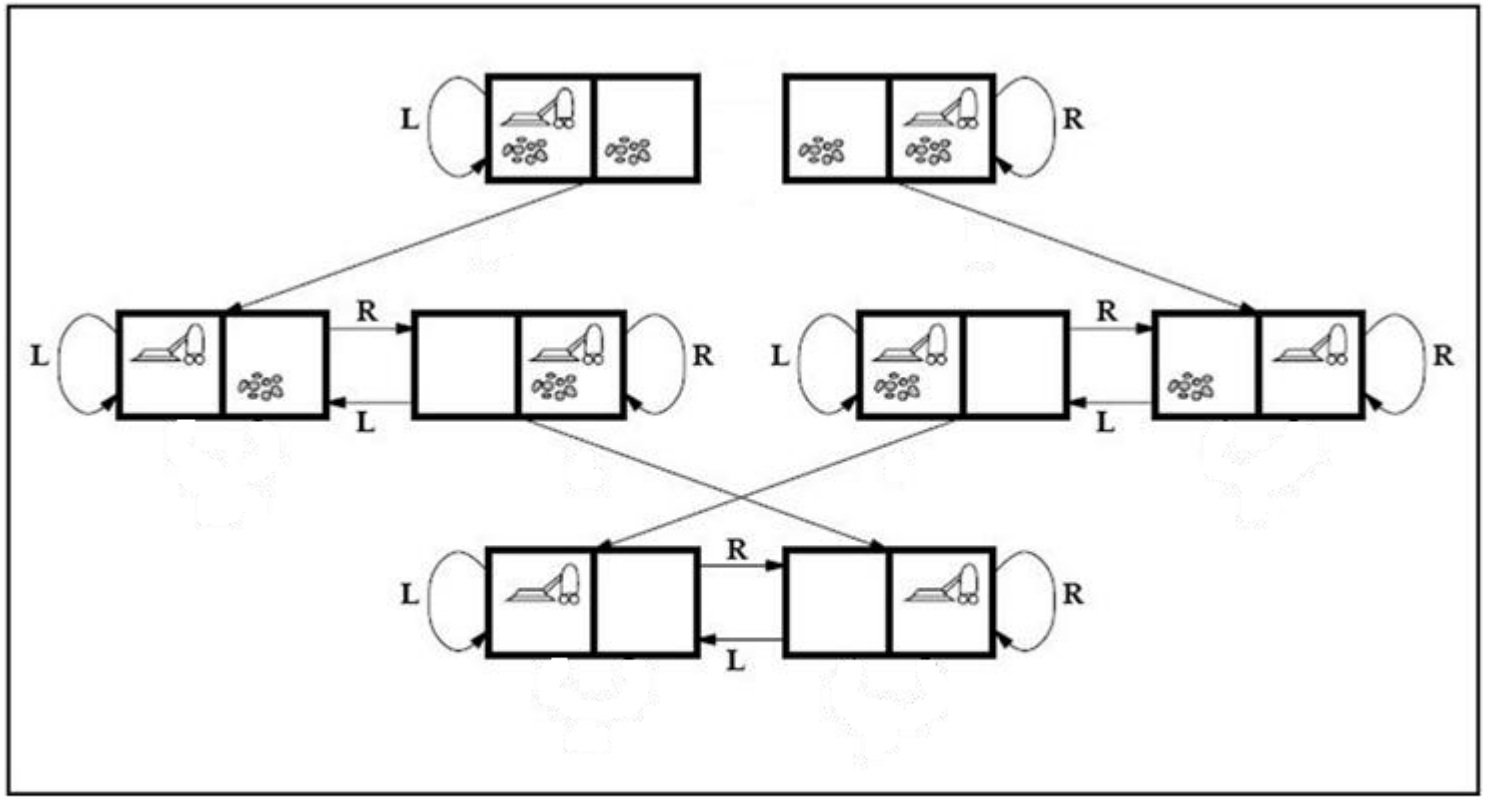
Cont.

- Solution
  - Path from the initial state to a goal state
- Search cost
  - Time and memory required to calculate a solution
- Path cost
  - Determines the expenses of the agent for executing the actions on a path
  - Sum of the costs of the individual actions in a path
- Total cost
  - Sum of search cost and path cost
  - Overall cost for finding a solution

Vacuum world state space graph

```
function REFLEX-VACUUM-AGENT( [location,status]) returns
action

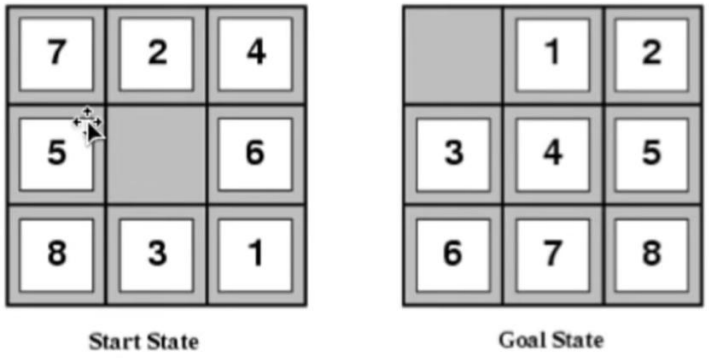
  if status = Dirty then return Clean
  else if location = A then return Right
  else if location = B then return Left
```



States: dirt and robot location
Actions: left, right, clean
Goal test: no dirt at all location
Path cost: 1 per action

There are 8 states

Example: 8-Puzzle



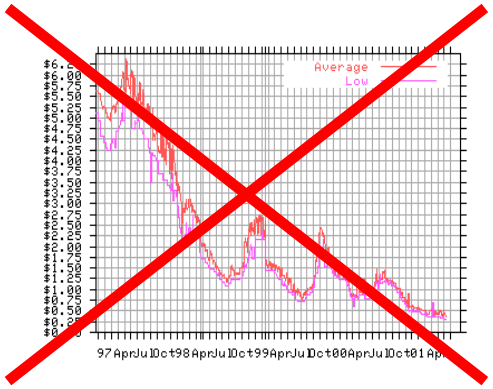
States: locations of tiles
Actions: move blank left, right, up, down
Goal test: Goal state
Path cost: 1 per action

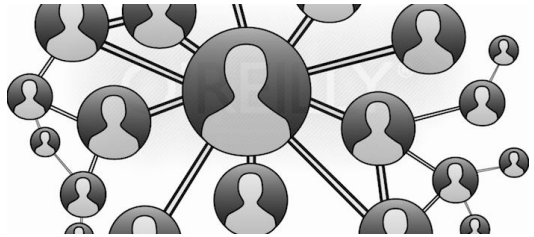
This is an NP-hard problem

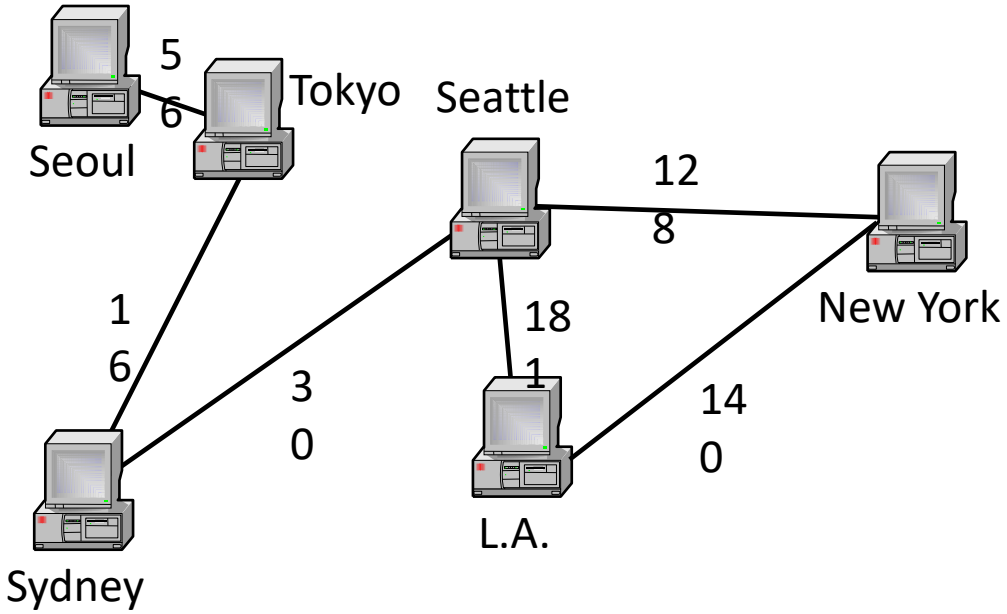
Motivation

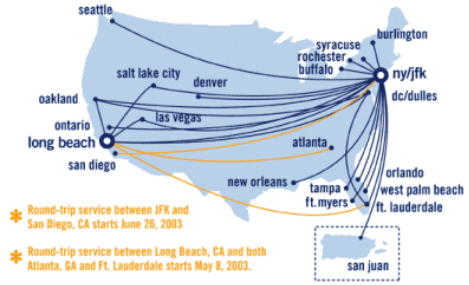
- Search algorithms are important methods for many approach to solve a problem.
- Search algorithms are the basis for many
  - optimization and
  - planning problems.

What is graph?



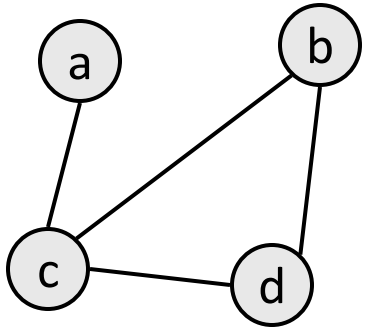






Graphs

- **graph**: A data structure containing:
  - a set of **vertices** *V*, *(sometimes called nodes)*
  - a set of **edges** *E*, where an edge represents a connection between 2 vertices.
    - Graph *G* = (*V*, *E*)
    - an edge is a pair (*v*, *w*) where *v*, *w* are in *V*

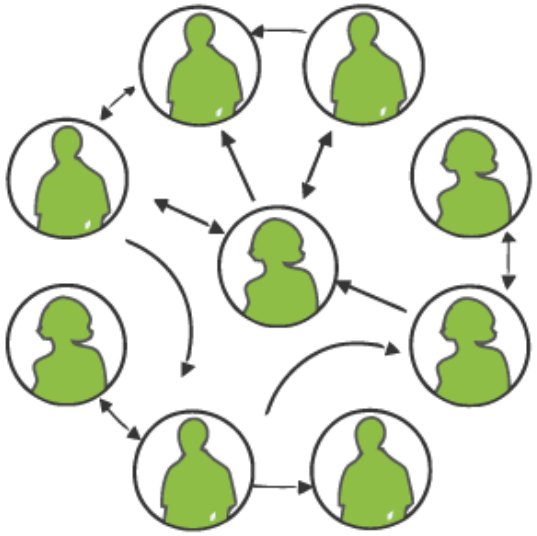


- the graph at right:
  - *V* = {a, b, c, d}
  - *E* = {(a, c), (b, c), (b, d), (c, d)}

- **degree**: number of edges touching a given vertex.
  - at right: a=1, b=2, c=3, d=2

Graph examples

- For each, what are the vertices and what are the edges?
  - Web pages with links
  - Methods in a program that call each other
  - Road maps (e.g., Google maps)
  - Airline routes
  - Facebook friends
  - Course pre-requisites
  - Family trees
  - Paths through a maze

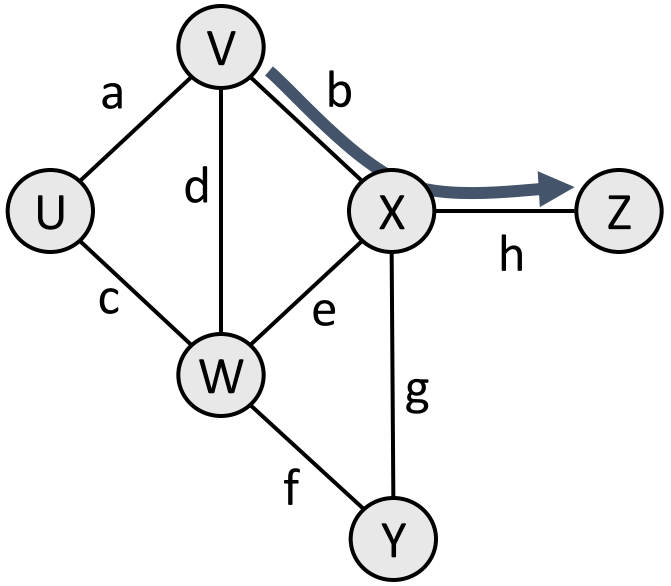


Paths

- **path**: A path from vertex *a* to *b* is a sequence of edges that can be followed starting from *a* to reach *b*.
  - can be represented as vertices visited, or edges taken
  - example, one path from *V* to *Z*: {b, h} or {V, X, Z}
  - What are two paths from U to Y?

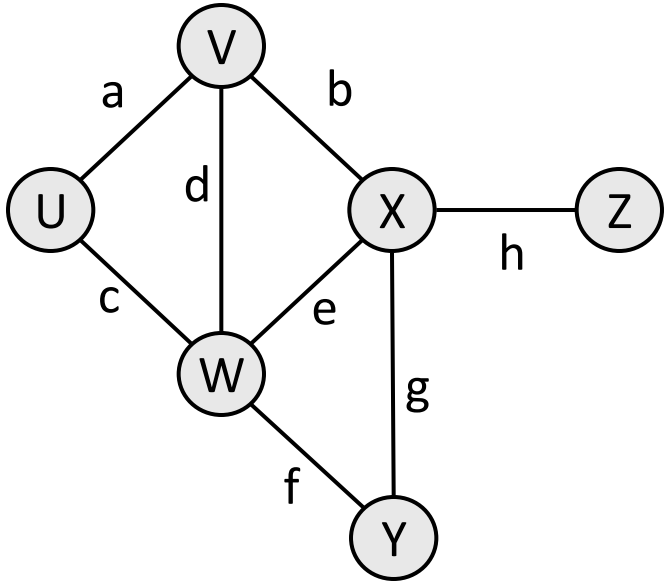
- **path length**: Number of vertices or edges contained in the path.

- **neighbor** or **adjacent***:* Two vertices connected directly by an edge.
  - example: V and X



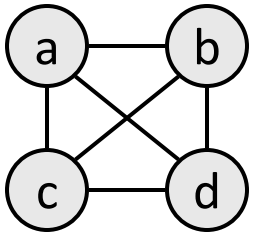
Reachability, connectedness

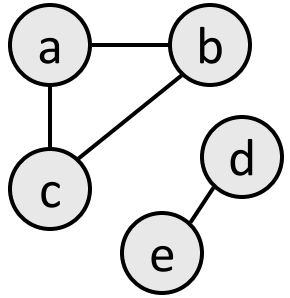
- **reachable**: Vertex *a* is *reachable* from *b* if a path exists from *a* to *b*.



- **connected**: A graph is *connected* if every vertex is reachable from any other.
  - Is the graph at top right connected?

- **strongly connected**: When every vertex has an edge to every other vertex.



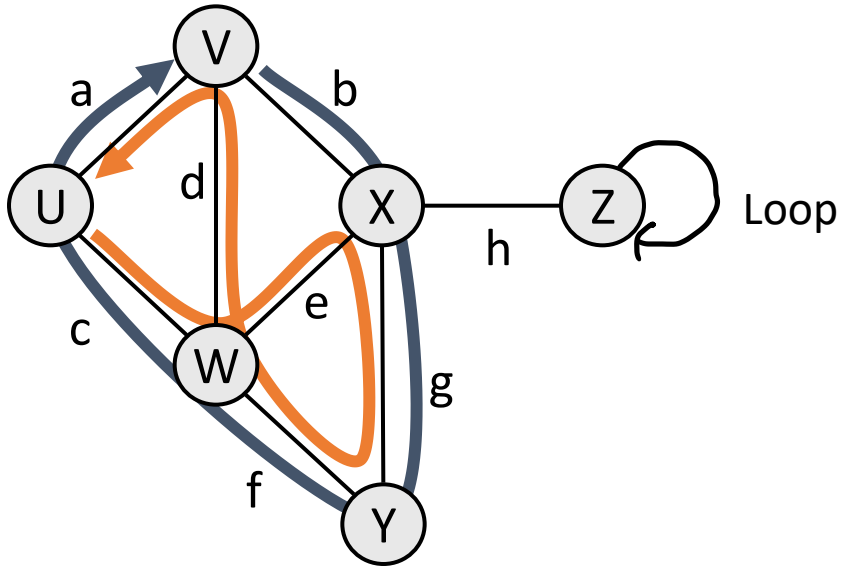


Loops and cycles

- **cycle**: A path that begins and ends at the same node.
  - example: {b, g, f, c, a} or {V, X, Y, W, U, V}.
  - example: {c, d, a} or {U, W, V, U}.

  - **acyclic graph**: One that does not contain any cycles.

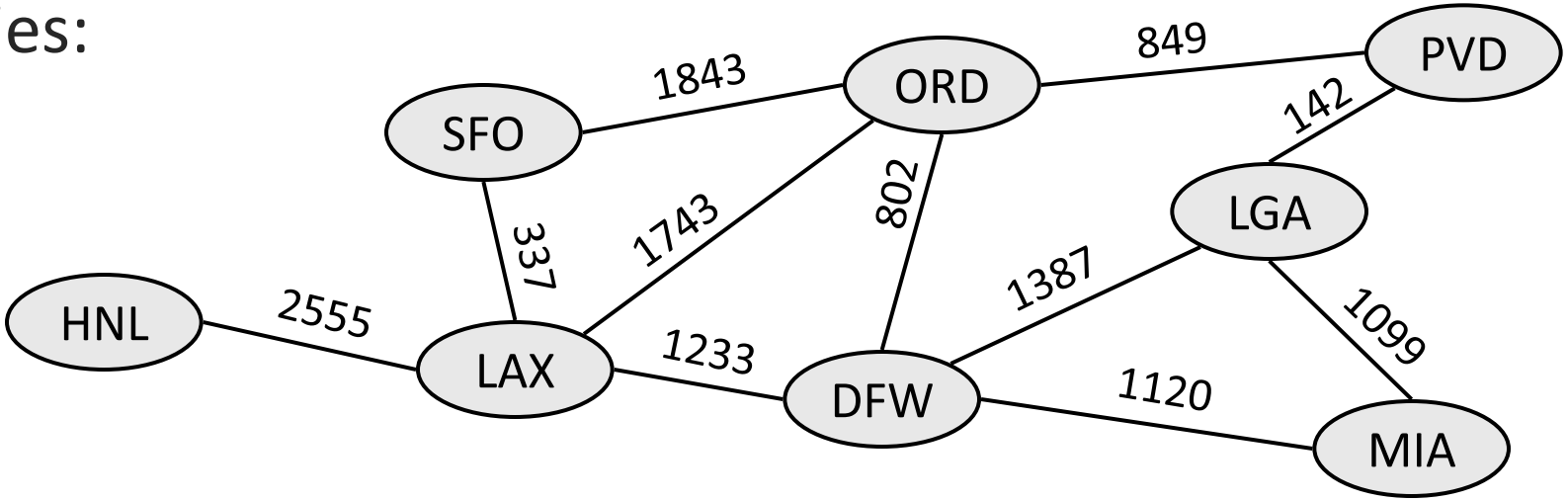
- **loop**: An edge directly from a node to itself.
  - Many graphs don't allow loops.



Weighted graphs

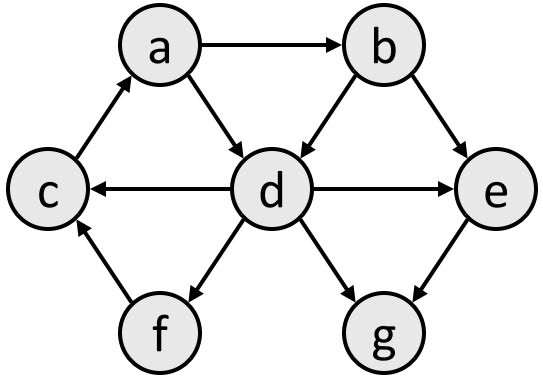
- **weight**: Cost associated with a given edge.
  - Some graphs have weighted edges, and some are unweighted.
  - Edges in an unweighted graph can be thought of as having equal weight (e.g. all 0, or all 1, etc.)
  - Most graphs do not allow **negative** weights.

- *example*: graph of airline flights, weighted by miles between cities:



Directed graphs

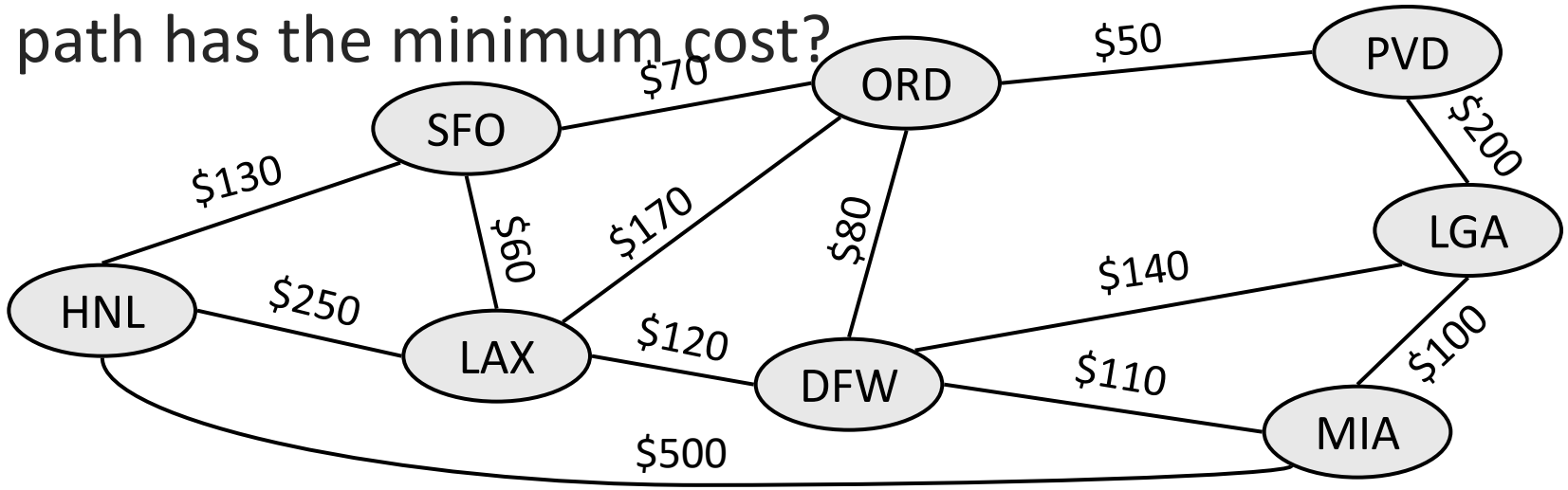
- **directed graph** ("digraph"): One where edges are *one-way* connections between vertices.
  - If graph is directed, a vertex has a separate in/out degree.
  - A digraph can be weighted or unweighted.
  - Is the graph below connected? Why or why not?



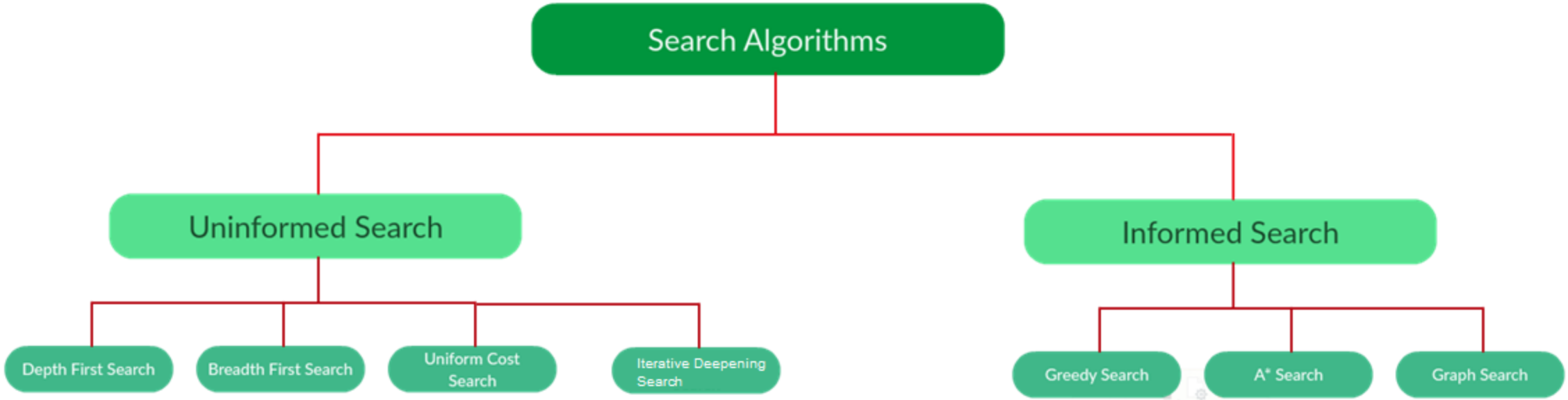
Searching for paths

- Searching for a path from one vertex to another:
  - Sometimes, we just want *any* path (or want to know there *is* a path).
  - Sometimes, we want to minimize path *length* (# of edges).
  - Sometimes, we want to minimize path *cost* (sum of edge weights).

- What is the shortest path from MIA to SFO? Which path has the minimum cost?



Types of search algorithms:



Uninformed search refers to a category of search algorithms that do not have any additional information or heuristics about the problem domain other than the information provided in the problem definition (e.g., the structure of the graph, the start and goal nodes, and possible transitions). These algorithms systematically explore the search space, making decisions solely based on the available data without any guidance about which direction might lead to the goal faster.

Informed search, also called **heuristic search**, refers to search algorithms that use additional domain-specific knowledge to guide the search process. These algorithms are designed to find solutions more efficiently by leveraging heuristics (estimates) to prioritize which nodes or paths to explore.

Uninformed Search Algorithms:

- also called Blind search.
- a class of general-purpose search algorithms.
- operates in brute force-way.
- do not have additional information about state or search space.
- generates the search tree without using any domain specific knowledge.

Depth First Search

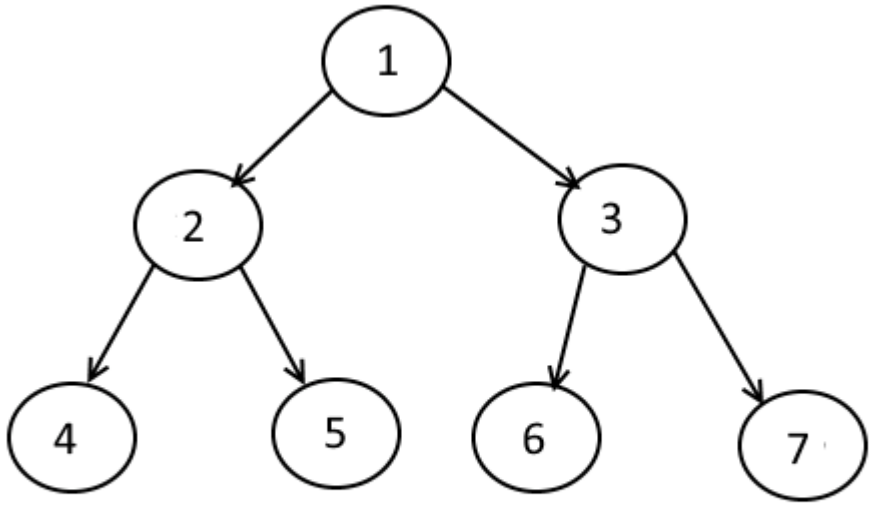
- **depth-first search** (DFS): Finds a path between two vertices by exploring each possible path as far as possible before backtracking.
  - Often implemented recursively.
  - Many graph algorithms involve *visiting* or *marking* vertices.

Steps:

1. Start at the root node and push it onto the stack.
2. Check for any adjacent nodes of the tree and select one node.
3. Traverse the entire branch of the selected node and push all the nodes into the stack.
4. Upon reaching the end of a branch (no more adjacent nodes) inth leaf node, move back by a single step and look for adjacent nodes of the n-1th node.
5. If there are adjacent nodes for the n-1th node, traverse those branches and push nodes onto the stack.

Example

Initial state



Goal state

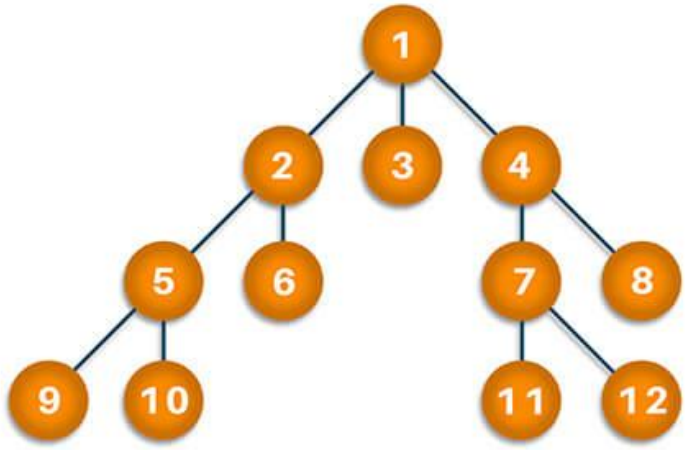
DFS - 1 2 4 5 3 6 7

preorder: Root-Left-Right

	Stack		Visited
push	1	pop	1
push	2 3	pop	2
push	4 5 3	pop	4
push	5 3	pop	5
push	3	pop	3
push	6 7	pop	6
push	7	pop	7

Example

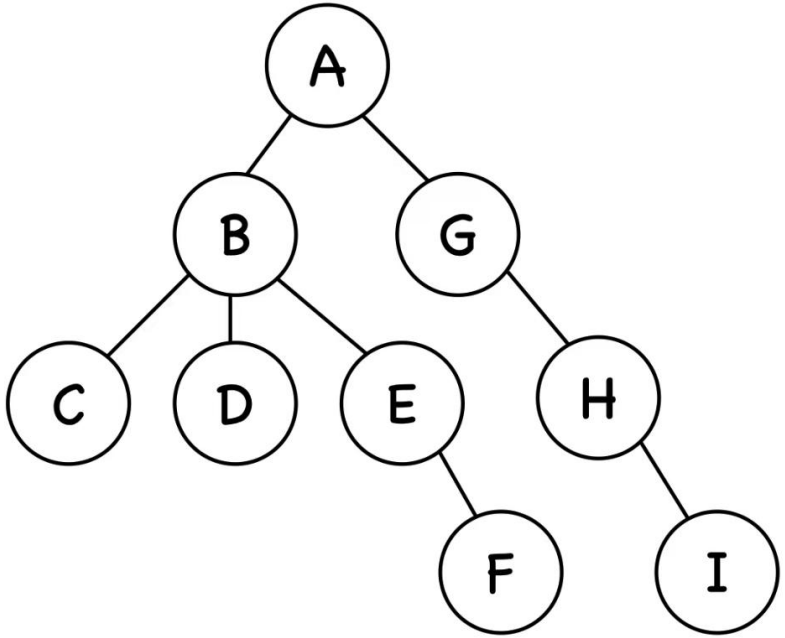
Initial state



Goal state

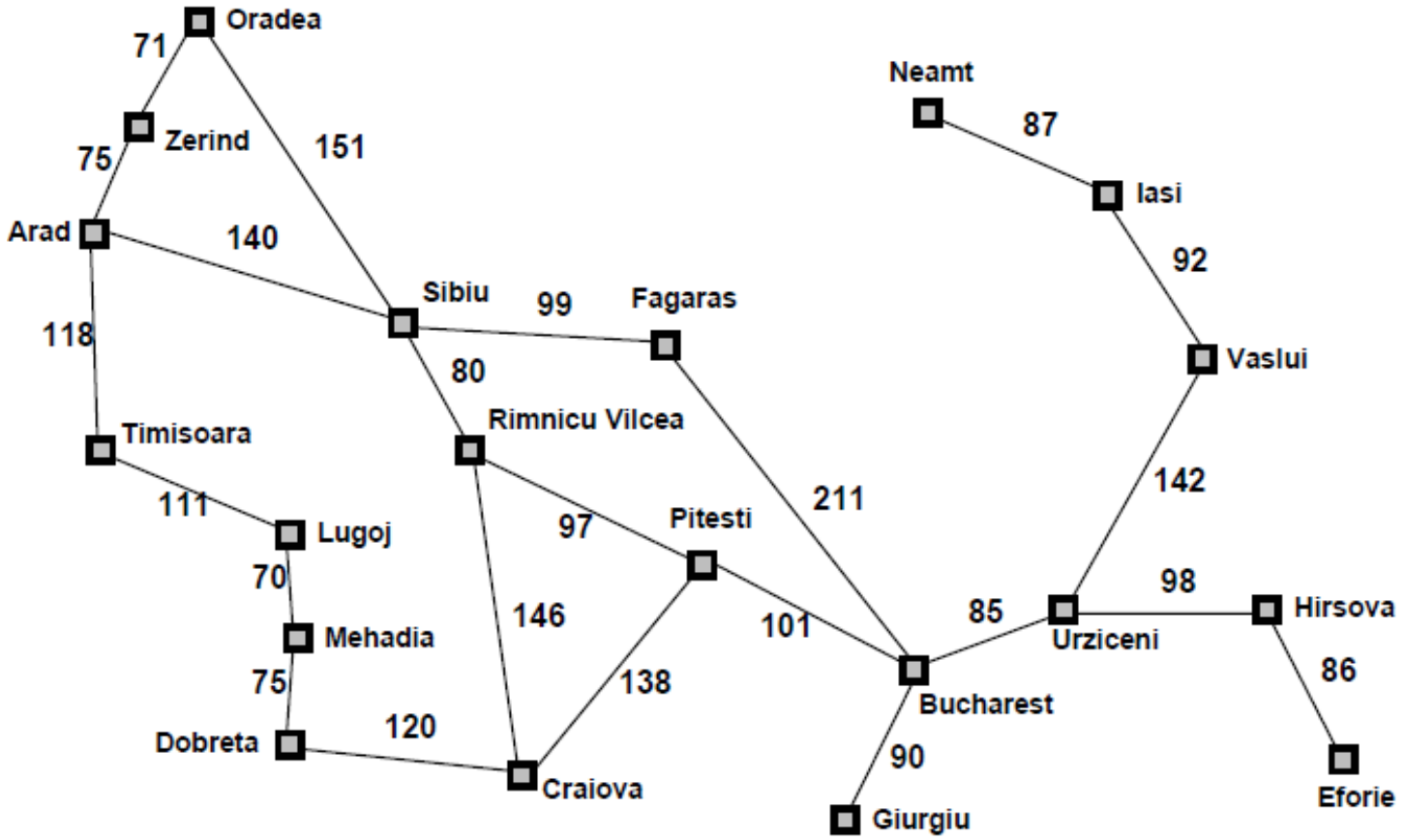
DEPTH FIRST SEARCH

Initial state



Goal state

Example: Bucharest Problem



Initial state: Arad
Destination: Bucharest

DFS is not suitable for finding the shortest path in **weighted graphs**.

DFS (stack = [(city, path)], LIFO)

Priority rule: City names are considered in alphabetical order.
Since DFS uses a LIFO stack, neighbors are pushed in reverse alphabetical order so that the alphabetically first city is visited first.

1. Start

stack = [(Arad, [Arad])]
visited = {}

2. Pop Arad → add Arad to visited

visited = {Arad}

Unvisited neighbors of Arad (alphabetical order): Sibiu, Timisoara, Zerind

stack = [
  (Sibiu, [Arad, Sibiu]),
  (Timisoara, [Arad, Timisoara]),
  (Zerind, [Arad, Zerind])
]

3. Pop Sibiu → add Sibiu to visited

visited = {Arad, Sibiu}

Neighbors of Sibiu:
Arad, Fagaras, Oradea, Rimnicu Vilcea

Arad is already visited.

Unvisited neighbors (alphabetical order): Fagaras, Oradea, Rimnicu Vilcea

stack = [
  (Fagaras, [Arad, Sibiu, Fagaras]),
  (Oradea, [Arad, Sibiu, Oradea]),
  (Rimnicu Vilcea, [Arad, Sibiu, Rimnicu Vilcea]),
  (Timisoara, [Arad, Timisoara]),
  (Zerind, [Arad, Zerind])
]

4. Pop Fagaras → add Fagaras to visited

visited = {Arad, Sibiu, Fagaras}

Neighbors of Fagaras: Bucharest, Sibiu

Sibiu is already visited.

Push.

stack = [
  (Bucharest, [Arad, Sibiu, Fagaras, Bucharest]),
  (Oradea, [Arad, Sibiu, Oradea]),
  (Rimnicu Vilcea, [Arad, Sibiu, Rimnicu Vilcea]),
  (Timisoara, [Arad, Timisoara]),
  (Zerind, [Arad, Zerind])
]

Solve the problem using DFS

5. Pop Bucharest → GOAL FOUND

Resulting DFS path:

Arad → Sibiu → Fagaras → Bucharest

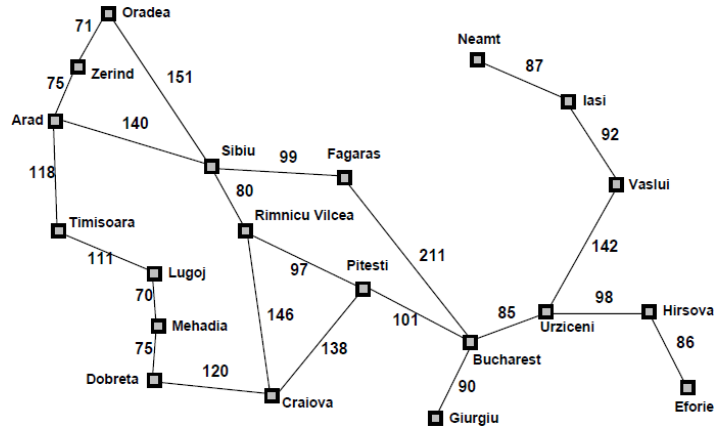
Path cost:

Arad → Sibiu = 140
Sibiu → Fagaras = 99
Fagaras → Bucharest = 211

Total cost = 140 + 99 + 211 = 450

DFS visited order:

Arad → Sibiu → Fagaras → Bucharest



DFS is deterministic given a neighbor order.

- You can explore the cities in different orders when running DFS.
- If you change the ordering, you change the search path.
- That's why in AI search, we usually care about **completeness** and **optimality** (where BFS or A* are better), because DFS alone might miss the "obvious" path depending on ordering

Breadth-First Search Algorithm

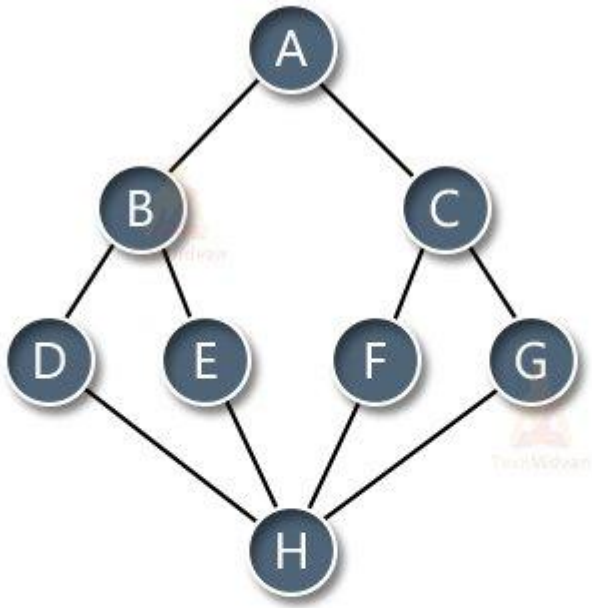
- **breadth-first search** (BFS): Finds a path between two nodes by taking one step down all paths and then immediately backtracking.
  - Often implemented by maintaining a queue of vertices to visit.

- BFS always returns the shortest path (the one with the fewest edges) between the start and the end vertices.

Cont.

- Breadth First Search is a traversal technique in which we traverse all the nodes of the graph in a breadth-wise motion.
- In BFS, we traverse one level at a time and then jump to the next level.
- In a graph, the traversal can start from any node and cover all the nodes level-wise.
- The BFS algorithm makes use of the queue data structure for implementation.
- In BFS, we always reach the final goal state (which was not always the case for DFS).

Example



Consider the following graph having 8 nodes named as A, B, C, D, E, F, G and H as shown:

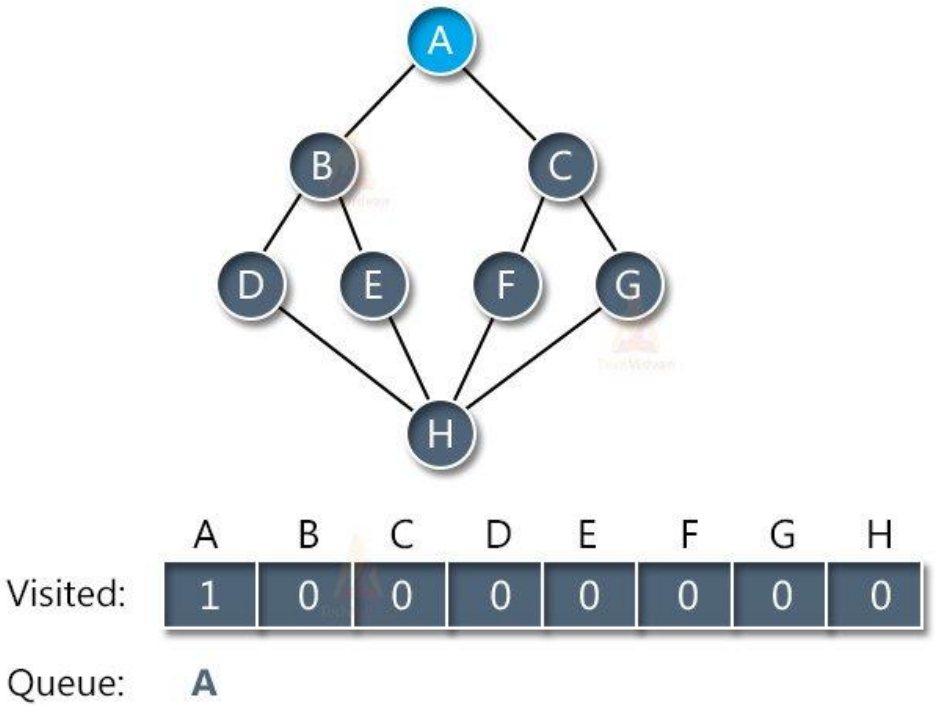
	A	B	C	D	E	F	G	H
Visited:	0	0	0	0	0	0	0	0

Queue: **Empty**

Cont.

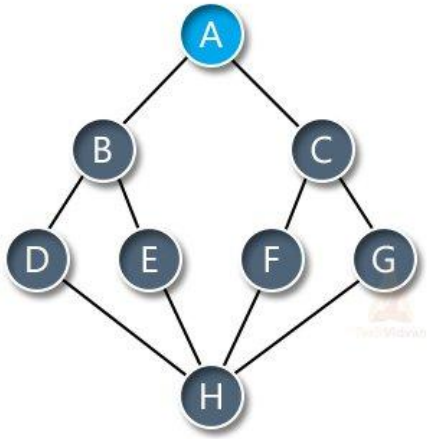
- Initially, the queue will be empty.

  Let us start traversing the graph from node A.
- Once we visit node A, we mark it as visited and also place it inside the queue as shown:



Cont.

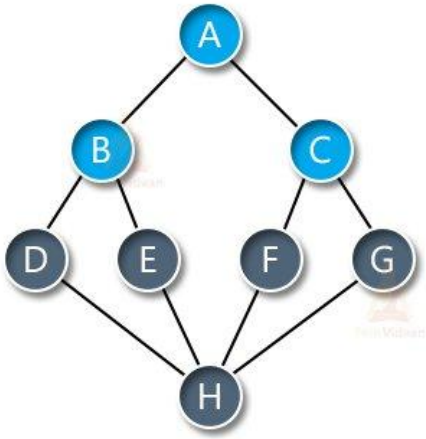
- The next step is to traverse its adjacent nodes i.e. B and C and place them inside the queue.
- When we place the adjacent node of A in the queue, we will remove A from the queue and display it in the output array as shown:



	A	B	C	D	E	F	G	H
Visited:	1	0	0	0	0	0	0	0

Queue: **Empty**

Output: **A**



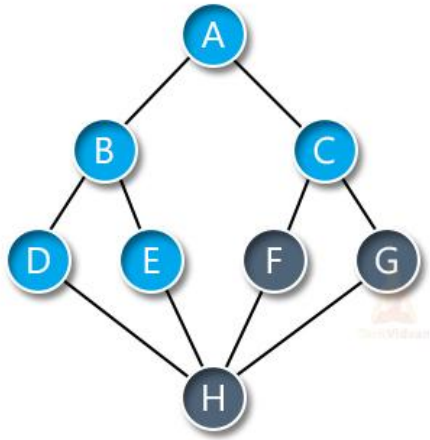
	A	B	C	D	E	F	G	H
Visited:	1	1	1	0	0	0	0	0

Queue: **B,C**

Output: **A**

Cont.

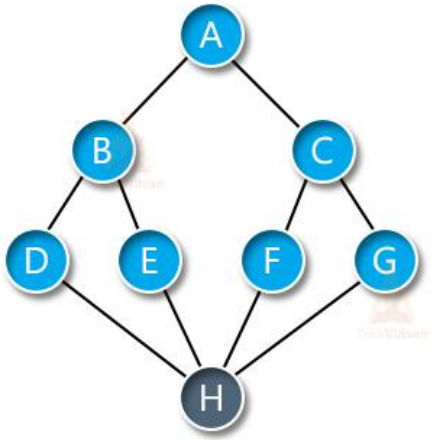
- Next, we don’t have any more adjacent nodes for A.
- Therefore, we will remove node B from the queue and place its adjacent nodes into the queue.
- Similarly, we will traverse the nodes of C and put them into the queue.



	A	B	C	D	E	F	G	H
Visited:	1	1	1	1	1	0	0	0

Queue: C,D,E

Output: A,B



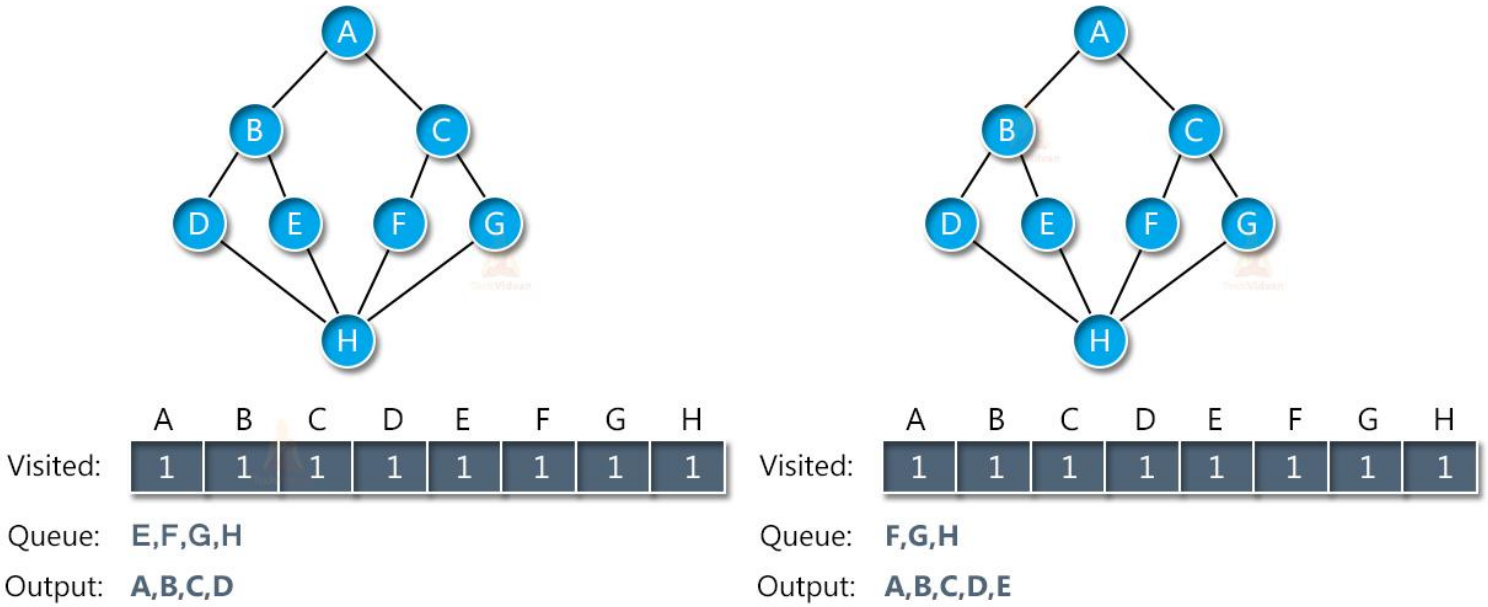
	A	B	C	D	E	F	G	H
Visited:	1	1	1	1	1	1	1	0

Queue: D,E,F,G

Output: A,B,C

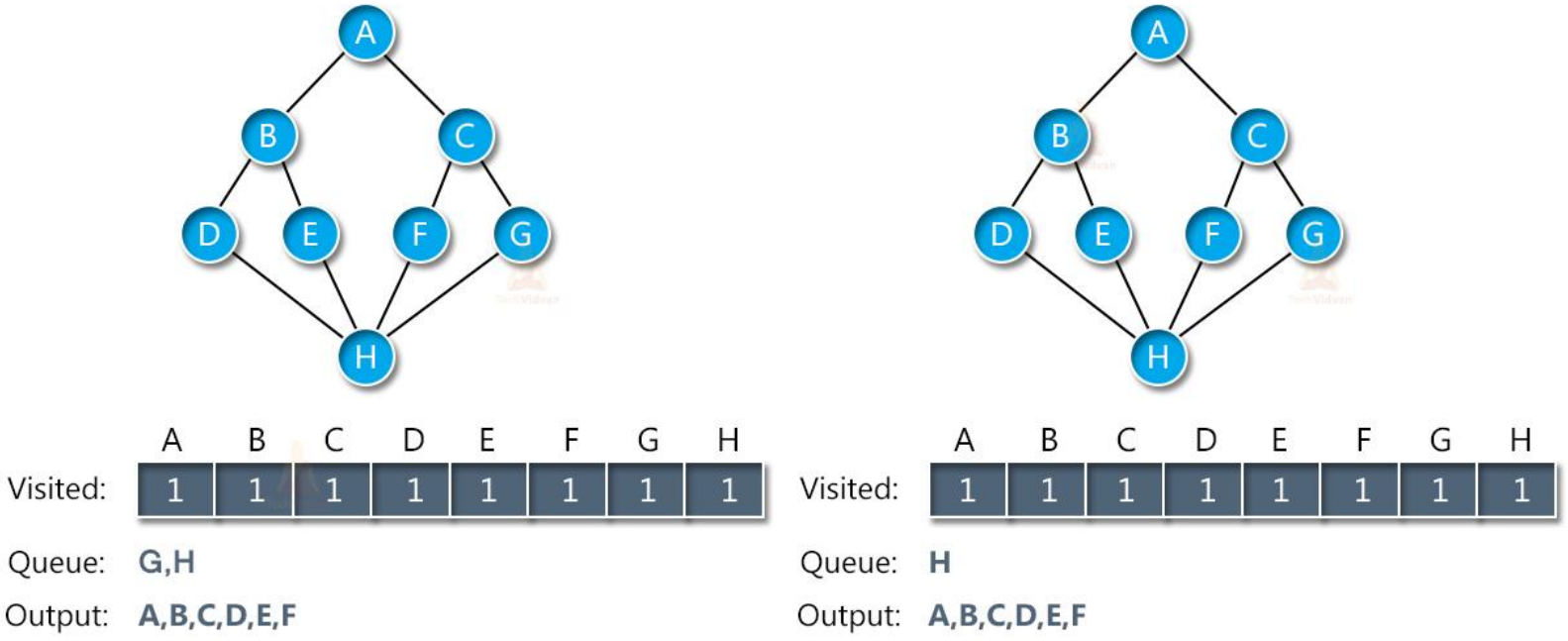
Cont.

- The next adjacent node is node H. thus, we will traverse that as well and place it in the queue.
- Once all the nodes have entered the queue, we will start removing them from the queue and putting them into the output array.



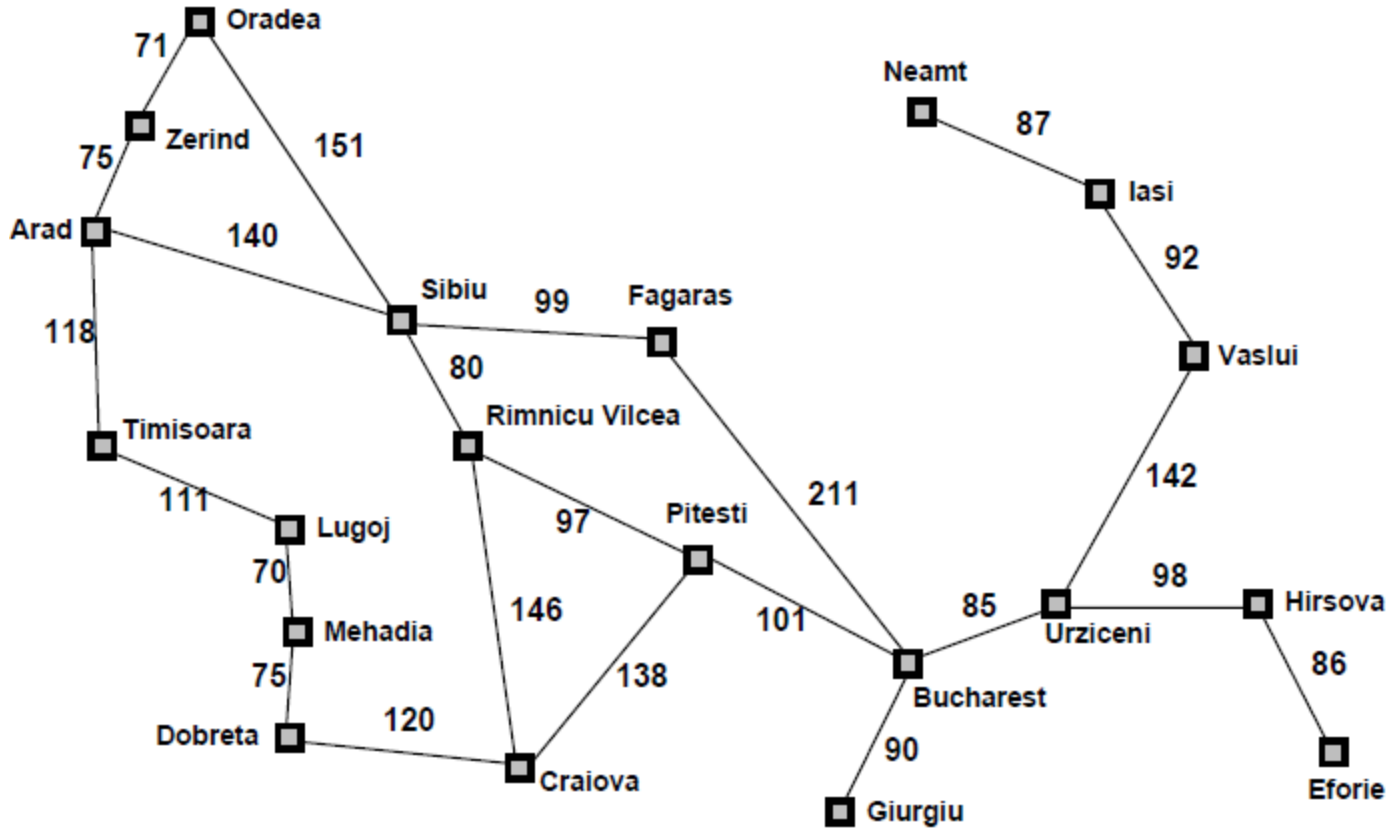
Cont.

- Thus, slowly the queue starts decreasing in size and the output array will be full as shown:



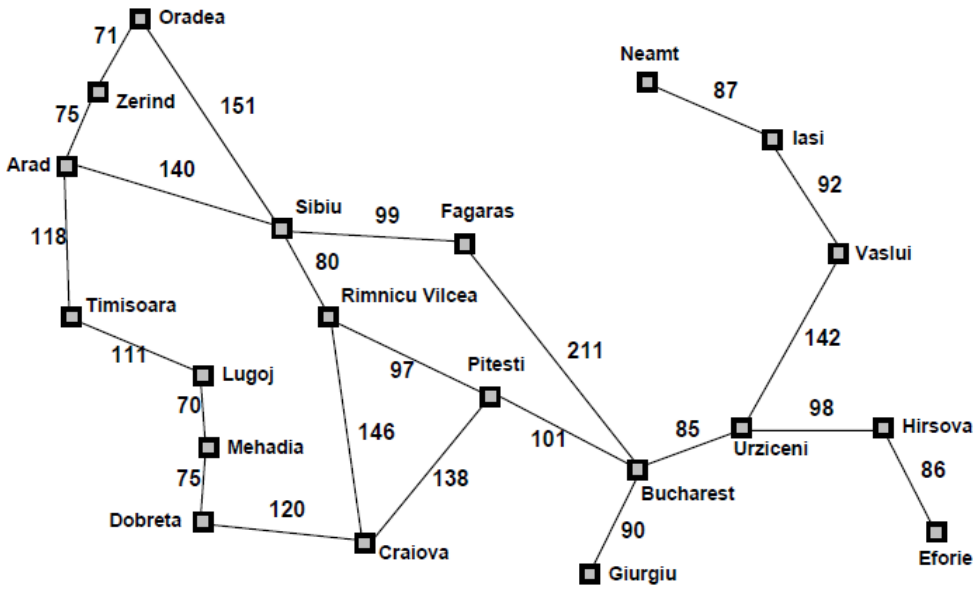
In this way, we will get the output of BFS to be A→B→C→D→E→F→G→H.

Example: Bucharest



Initial state: Arad
Destination: Bucharest

Solution



BFS: Arad → Bucharest
Priority: Alphabetical order of city names

Step 1:
Start at Arad.
Visited: Arad

Neighbors of Arad:
- Sibiu
- Timisoara
- Zerind

Queue: **[Sibiu, Timisoara, Zerind]**

Step 2: Visit Sibiu.

New neighbors:
- Fagaras
- Oradea
- Rimnicu Vilcea

Add the new cities to the queue:
[Timisoara, Zerind, Fagaras, Oradea, Rimnicu Vilcea]

Visited:
Arad → Sibiu

Step 3: Visit Timisoara.

New neighbor:
- Lugoj

Visited:
Arad → Sibiu → Timisoara

Add the new cities to the queue: **[Zerind, Fagaras, Oradea, Rimnicu Vilcea, Lugoj]**
Step 4: Visit Zerind.

Neighbor:
- Oradea

Oradea is already discovered, so we do not add it again.

Visited:
Arad → Sibiu → Timisoara → Zerind

Add the new cities to the queue: **[Fagaras, Oradea, Rimnicu Vilcea, Lugoj]**

Step 5: Visit Fagaras.

New neighbor:
- Bucharest

Add the new cities to the queue: **[Oradea, Rimnicu Vilcea, Lugoj Bucharest]**

Bucharest is discovered, but BFS must finish checking
the remaining cities at the current depth first.

Step 6: Visit Oradea.

No new cities.

Add the new cities to the queue: **[Rimnicu Vilcea, Lugoj, Bucharest, Mehadia]**

Step 7: Visit Rimnicu Vilcea.

New neighbors:
- Craiova
- Pitesti

Add the new cities to the queue: **[Lugoj, Bucharest, Mehadia, Craiova, Pitesti]**

Step 8: Visit Lugoj.

No new cities.

Add the new cities to the queue: **[Bucharest, Mehadia, Craiova, Pitesti]**

Step 9:
Visit Bucharest.

GOAL FOUND!

BFS VISITING ORDER:

Arad
→ Sibiu
→ Timisoara
→ Zerind
→ Fagaras
→ Oradea
→ Rimnicu Vilcea
→ Lugoj
→ Bucharest

SOLUTION PATH:

Arad → Sibiu → Fagaras → Bucharest

Depth:

Arad	Depth 0
Sibiu	Depth 1
Fagaras	Depth 2
Bucharest	Depth 3

Therefore, BFS finds Bucharest at depth 3.

Final Answer:
Arad → Sibiu → Fagaras → Bucharest

* **DFS and BFS** are not suitable for finding the shortest path in **weighted graphs**.

A* Search algorithm

- The A search algorithm* is a widely used and powerful algorithm for pathfinding and graph traversal.

- It is used in various applications such as navigation systems, game AI, and robotics.

- A* combines features of Dijkstra's Algorithm and Greedy Best-First Search to efficiently find the shortest path from a starting node to a goal node.

How A* Search Algorithm Works

- **Initialization**:
- **Open List**: A priority queue (often implemented as a min-heap) that contains nodes to be evaluated. Initially, this list contains only the start node.
- **Closed List**: A set of nodes that have already been evaluated.

Node Evaluation:

- The algorithm repeatedly selects the node with the lowest cost estimate from the open list. The cost estimate is given by f(n)=g(n)+h(n) where:
  - g(n): The cost from the start node to node n (known as the **path cost**).
  - h(n): A heuristic estimate of the cost from node n to the goal (known as the **heuristic cost** or **estimated cost**).
  - f(n): The total estimated cost of the cheapest solution through node n.

Expansion:

- The selected node is moved from the open list to the closed list.
- The algorithm examines each neighboring node of the current node. For each neighbor:
  - If it is the goal node, the algorithm terminates successfully, reconstructing the path from the start to the goal node.
  - If it is not in the closed list, calculate its *g*, *h*, and *f* values.
  - If it is not in the open list, add it with its cost estimates.
  - If it is already in the open list but the new path to it is cheaper, update its cost estimates and path information.

Termination

- The algorithm terminates when the goal node is selected.

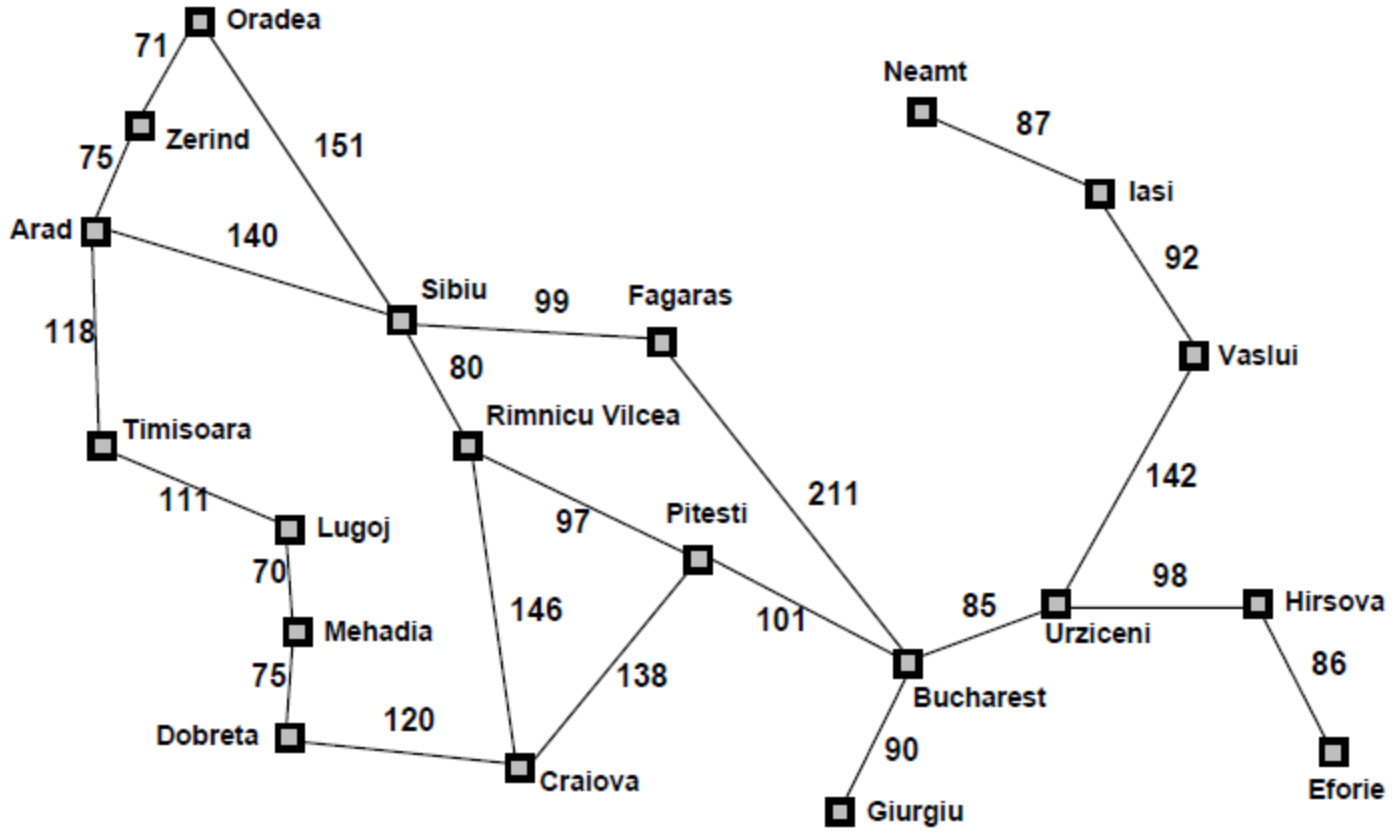
Advantages of A* Search Algorithm

- **Optimality**: A* is guaranteed to find the shortest path if the heuristic function is admissible and consistent.

- **Efficiency**: By combining the best features of Dijkstra's Algorithm and Greedy Best-First Search, A* is generally more efficient than using either algorithm alone.

- **Flexibility**: The algorithm can be adapted to various types of graphs and search spaces by choosing appropriate heuristic functions.

Example: Bucharest



Initial: Arad
Destination: Bucharest

Heuristic to Bucharest

City	Heuristic to Bucharest (km)
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu_Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

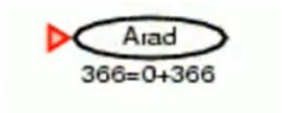
It is **not calculated from the road edges on the search graph**.

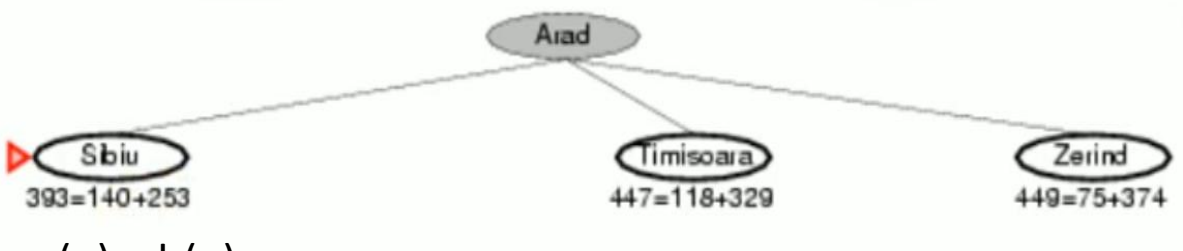
Instead, we use the geographical positions of **Sibiu and Bucharest** and calculate their approximate straight-line distance:

$$h(\text{Sibiu}) = \text{straight-line distance}(\text{Sibiu}, \text{Bucharest})$$

If you have coordinates (x_1, y_1) and (x_2, y_2), a simplified Euclidean calculation is:

$$\boxed{h(n) = \sqrt{(x_n - x_{goal})^2 + (y_n - y_{goal})^2}}$$

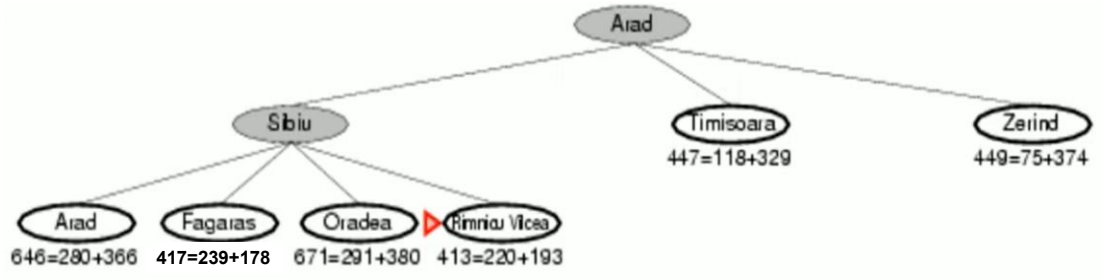


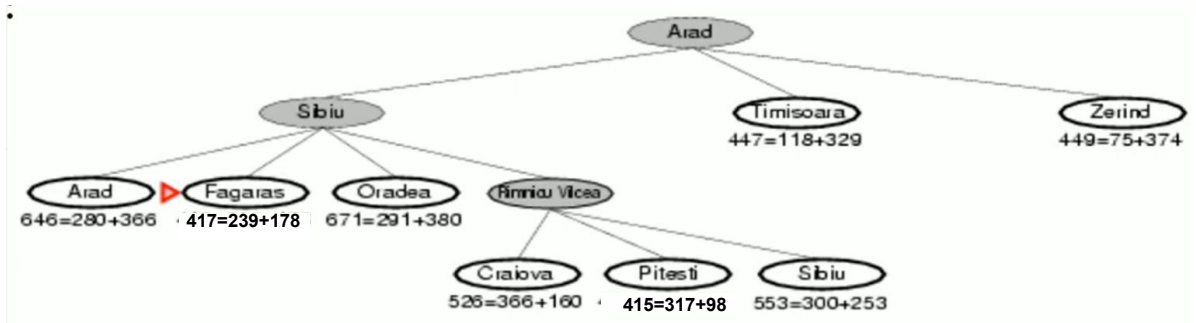


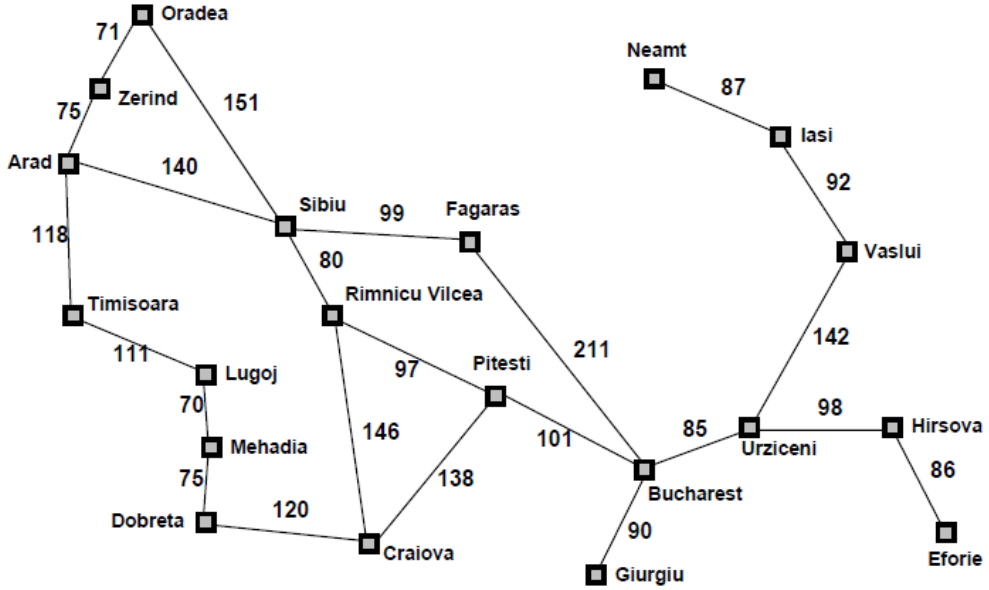
g(n) + h(n)

g(n) + h(n)

g(n) + h(n)

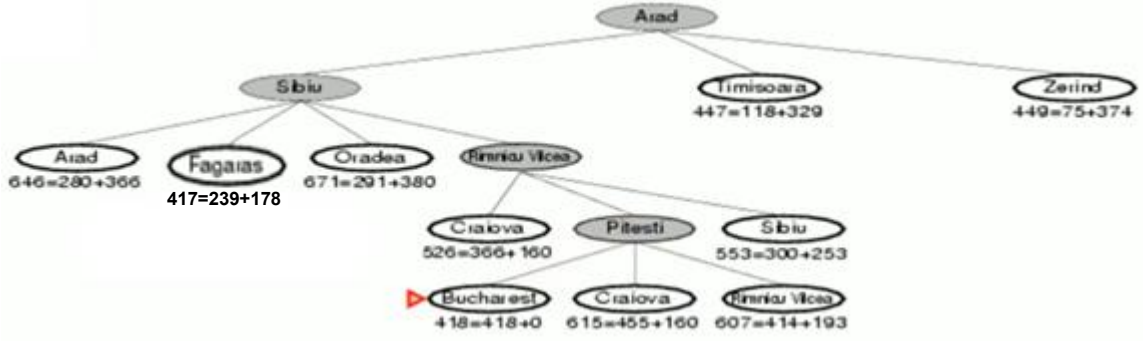


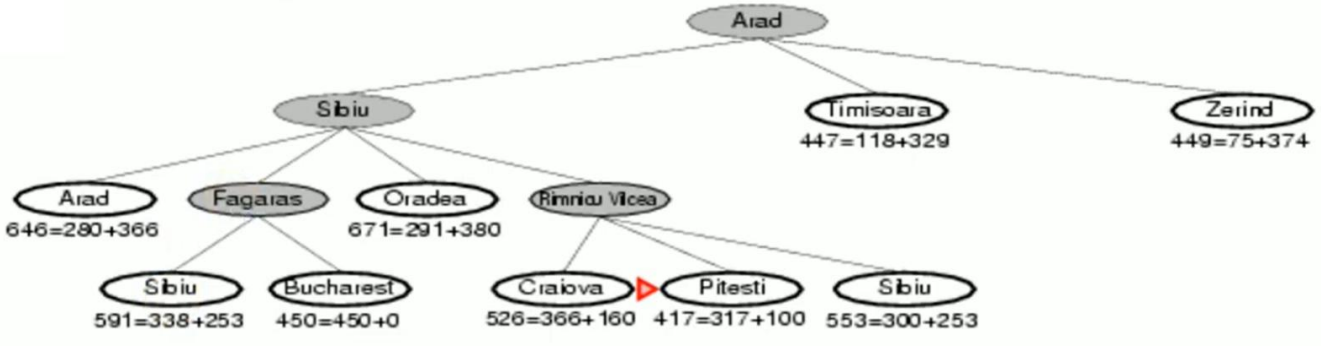




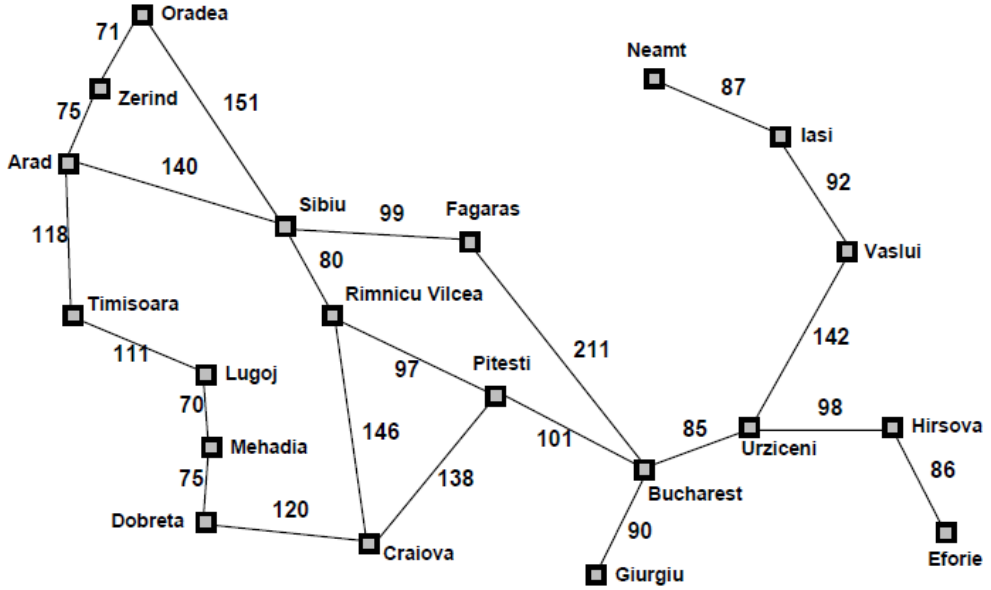
City	Heuristic to Bucharest (km)
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu_Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Bucharest first appears in this step





This is the final solution. And the final path is



City	Heuristic to Bucharest (km)
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu_Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Solution

- The path Arad -> Sibiu -> Fagaras -> Bucharest with a total cost of 450 is a valid path but not necessarily the shortest.
- The verified shortest path is Arad -> Sibiu -> Rimnicu_Vilcea -> Pitesti -> Bucharest with a cost of 418.

Thank you for listening